

AHMADU BELLO UNIVERSITY, ZARIA

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

COEN838: ADVANCED SOFTWARE ENGINEERING

Project Examination, Question 4

OFFLINE-FIRST CRYPTOGRAPHIC QR IDENTITY WALLET

FOR RURAL PUBLIC SERVICES

SOFTWARE DESIGN DOCUMENT

Document QRW-SDD-02 · Version 1.0

BY

ALIYU SARKI ILIYASU

P25EGCP8040

November 2026

DOCUMENT CONTROL

Document details

Item	Detail
Document	Software Design Document (QRW-SDD-02)
Project	Offline-First Cryptographic QR Identity Wallet for Rural Public Services
Course	COEN838 Advanced Software Engineering, Ahmadu Bello University, Zaria
Author	Aliyu Sarki Iliyasu (P25EGCP8040)
Version / date	1.0 · November 2026
Source code	https://github.com/sarki94/qr-identity-wallet
Related documents	QRW-SRS-01 Requirements · QRW-SEC-03 Security · QRW-TST-04 Test Plan & Report

Revision history

Version	Date	Change
1.0	4 Nov 2026	First complete design document

TABLE OF CONTENTS

1. INTRODUCTION....1
1.1 Purpose....1
1.2 Scope....1
1.3 Design Stakeholders and Concerns....1
1.4 References....1
2. DESIGN CONSIDERATIONS....3
2.1 Architectural Drivers....3
2.2 Design Decisions....3
3. ARCHITECTURAL DESIGN....5
3.1 Architectural Style and Justification....5
3.2 Component View....5
3.3 Deployment View....6
3.4 Inter-Component Communication....7
3.5 Offline-First Design....8
4. DATA DESIGN....9
4.1 Data Model....9
4.2 Data Dictionary....9
4.3 Credential and Message Formats....10
5. COMPONENT DESIGN....12
5.1 Cryptographic Core Library....12
5.1.1 Verification algorithm....12
5.1.2 Atomic re-issue and publication....12
5.2 Back-End Services....13
5.3 Web Application....13
6. INTERFACE DESIGN....14
7. SCALABILITY AND RESILIENCE....15
7.1 Scalability....15
7.2 Resilience....15
8. SECURITY DESIGN SUMMARY....17
9. DESIGN TRACEABILITY....18

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) describes how the Offline-First Cryptographic QR Identity Wallet is built to satisfy the requirements in QRW-SRS-01. It records the architectural style and the reasons for choosing it, the components and their responsibilities, the data model, the communication patterns between components, the detailed design of the security-critical algorithms, and the provisions for scalability and resilience. Its structure follows the viewpoint approach of IEEE 1016, the standard for software design descriptions (IEEE, 2009).

1.2 Scope

The design covers the cryptographic core library, the four back-end services and the mock NIMC service, and the Progressive Web App with its Verifier, Wallet, Issuer Portal and Audit screens. Security design is summarised in Section 8 and treated fully in QRW-SEC-03.

1.3 Design Stakeholders and Concerns

Table 1.1 lists who reads this design and what each needs it to answer.

Table 1.1: Design stakeholders and concerns

Stakeholder	Concern	Addressed in
Examiners	Is the architectural style justified? Are communication patterns, data model and scalability considered?	Sections 3, 4, 7
Programme sponsor	Will it work where the network fails? Can it be trusted?	Sections 3.5, 7, 8
Developer / maintainer	Where does each responsibility live? How do the parts talk?	Sections 3, 5, 6
Security reviewer	Where are the keys? What does each component trust?	Sections 5.1, 8; QRW-SEC-03

1.4 References

Fältström, P., Ljunggren, F., & van Gulik, D.-W. (2022). *The Base45 data encoding* (RFC 9285). Internet Engineering Task Force. https://doi.org/10.17487/RFC9285

IEEE. (2009). *IEEE standard for information technology — Systems design — Software design descriptions* (IEEE Std 1016-2009). Institute of Electrical and Electronics Engineers.

Josefsson, S., & Liusvaara, I. (2017). *Edwards-curve digital signature algorithm (EdDSA)* (RFC 8032). Internet Engineering Task Force. https://doi.org/10.17487/RFC8032

Newman, S. (2021). *Building microservices* (2nd ed.). O'Reilly Media.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

2. DESIGN CONSIDERATIONS

2.1 Architectural Drivers

Four drivers from the requirements shaped the design more than any others:

1. **Verification must work with no network** (problem statement; FR-13, FR-16 to FR-22, NFR-P3). The trust decision therefore has to be computable on the agent's phone from data downloaded earlier.
2. **Credentials must be unforgeable and uncopyable** (FR-16, FR-17, FR-23; STRIDE spoofing and tampering). This rules out database look-ups as the basis of trust and points to digital signatures with holder binding.
3. **The signing key is the most valuable asset** (NFR-S1). Whatever the architecture, only one small component may hold it.
4. **A solo developer must deliver a working, tested system in days.** Every extra moving part (broker, orchestrator, separate codebases) costs time without earning marks.

2.2 Design Decisions

The main decisions, with their rationale, are summarised in Table 2.1. The full log with defence notes is kept in the project repository.

Table 2.1: Key design decisions

ID	Decision	Rationale
D2a	Two assurance levels on one credential format: L2 smartphone wallet, L1 paper wallet	Elicitation found shared phones and feature phones (F2, F3); one format keeps one verifier code path.
D3	Four services: Auth, Issuer, Trust & Revocation, Audit & Sync, plus a mock NIMC	Split on trust and availability boundaries, not on screens.
D4	FastAPI services; one React PWA for every screen	Reuses the developer's existing stack; a PWA runs offline and installs on phones.
D6	No proxy or delegated presentation	Programme policy observed in Kura (F1).
D8	Atomic re-issue in one database transaction	A farmer never has two active credentials, nor none.
D9	Sign the exact payload bytes; QR = "QRW1:" + Base45(zlib(signature ‖ payload)); typed messages	No canonicalisation mismatch between Python and the browser; dense QR alphanumeric mode; domain separation.
D11	Issuer signs the revocation list; Trust only verifies and publishes it	Keeps the signing key in one place; Trust cannot forge or roll back a list.
D12	Database per service; EdDSA-signed staff tokens verified with Auth's public key	No service reads another's data; services can check tokens but not mint them.
D13	Store NINs only as a keyed hash plus a masked copy	A stolen Issuer database reveals no NIN.
D14	Strict RFC 8032 verification in the browser; one HTTPS origin via a proxy	Browser and server accept exactly the same signatures; phones need HTTPS for the camera.

3. ARCHITECTURAL DESIGN

3.1 Architectural Style and Justification

The system combines three styles: an **offline-first edge client** for verification, a set of **service-oriented back-end services** each owning its own data, and **asynchronous store-and-forward synchronisation** between them. Table 3.1 compares this with the alternatives considered.

Table 3.1: Architectural alternatives

Style	Strengths	Weaknesses for this problem	Decision
Single monolith with a central database	Simplest to build and deploy	Verification would need the server, which is exactly what fails in rural areas; one breach exposes the signing key and all data	Rejected
Online microservices with a message broker (e.g. Kafka) and orchestrator	Industry-scale throughput and decoupling	Still requires connectivity at verification time; broker and orchestration add days of work and operational burden without serving any requirement	Rejected
Federated, per-state deployments	Local autonomy	Duplicates issuance and trust management; verifiers would need many trust anchors	Deferred (future scaling path)
Offline-first edge verification + service-oriented back end (chosen)	Verification needs no network; trust boundaries isolate the key; services stay small and testable	Revocation reaches offline points only on their next sync (managed as a risk)	Adopted

The decisive argument is that the problem statement makes network failure the normal case. Any architecture that consults a server at the moment of verification fails the requirement, however well it scales. Signatures move the trust decision to the edge: the server's role shrinks to issuing, publishing and collecting, which can all tolerate delay.

3.2 Component View

Figure 3.1 shows the components and how they connect, and Table 3.2 lists their responsibilities.

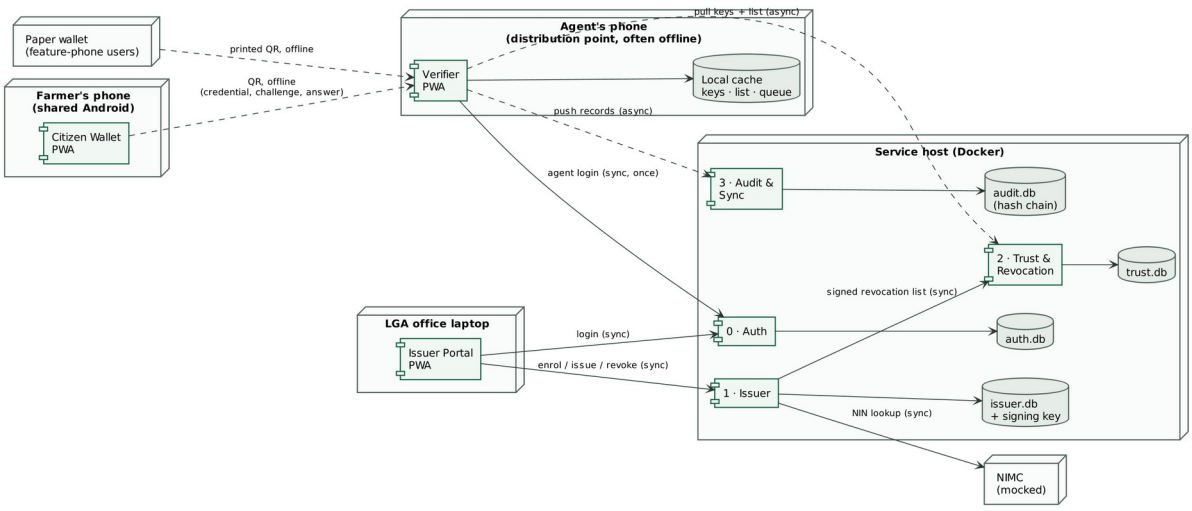


Figure 3.1: Component and connector view

Table 3.2: Component responsibilities

Component	Responsibility	Owns data	Holds secrets
Auth (8001)	Staff login, password hashing (scrypt), lockout, EdDSA tokens, staff administration	auth.db: users	Auth private key
Issuer (8002)	NIMC look-up, enrolment, credential signing, atomic re-issue, revocation, signing and publishing the revocation list	issuer.db: farmers, credentials, revocation_state	Issuer signing key; NIN pepper; service key
Trust & Revocation (8003)	Publishes issuer public keys and the latest signed revocation list; verifies pushed lists and refuses rollback	trust.db: published_list	Service key only
Audit & Sync (8004)	Receives queued verification records, chains them by hash, verifies the chain, flags anomalies	audit.db: records	None
Mock NIMC (8005)	Returns fictional NIN records (stand-in for NIMC)	Bundled JSON	None
Web app (PWA)	Verifier, Wallet, Issuer Portal, Audit and Guided Demo screens; offline verification in the browser	Browser storage on each device	Wallet: PIN-sealed device keys
qrw_core (library)	Pure functions: signing, verification, encoding, challenge–response, revocation list, hash chain	None	None

3.3 Deployment View

The deployment is shown in Figure 3.2. All five services are built from one Docker image and started as five containers by Docker Compose; a one-off container creates the key material in a shared, encrypted volume. Each service has its own data volume. The web application is a static build served over HTTPS by a server that also proxies

/api/<service> to the services, so phones talk to a single origin. For laptops without Docker, a launcher script starts the same five services as local processes.

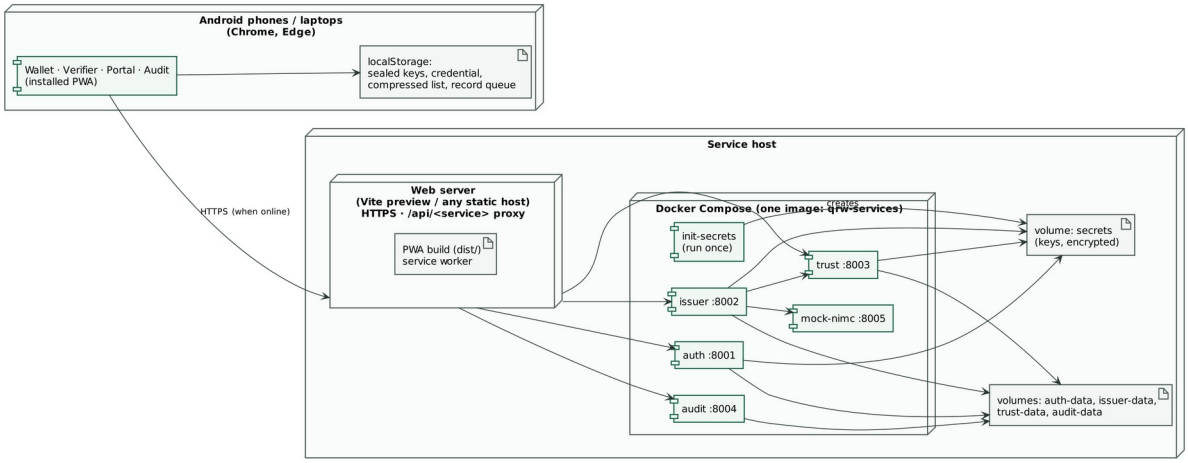


Figure 3.2: Deployment view

For production the same structure would sit behind a TLS-terminating reverse proxy, with PostgreSQL databases (selected by setting QRW_<SERVICE>_DATABASE_URL) and the issuer key held in a hardware security module (assumption AS2 in QRW-SRS-01).

3.4 Inter-Component Communication

Each interaction uses the pattern that matches its timing needs. Table 3.3 justifies each choice; the sequence diagrams in QRW-SRS-01 (Figures 4.2 to 4.4) show them in detail.

Table 3.3: Communication patterns

Interaction	Pattern	Protocol	Why
Portal → Auth, Portal → Issuer	Synchronous request–response	HTTPS REST/JSON	The officer is online and needs an immediate answer to continue enrolment.
Issuer → Mock NIMC	Synchronous	HTTP (internal)	Enrolment cannot proceed without the NIN record.
Issuer → Trust (publish list)	Synchronous with retry	HTTP + service key	Publication should be immediate, but a Trust outage must not block revocation; the Issuer records the unpublished version and republishes later.
Wallet ↔ Verifier	Offline, device to device	QR codes (four formats)	No network exists at the distribution point.
Trust → Verifier (keys, list)	Asynchronous pull	HTTPS GET, gzip	The verifier fetches whenever it has a signal; freshness is managed by list-age rules.
Verifier → Audit (records)	Asynchronous push, store-and-forward, batched	HTTPS POST	Records accumulate offline and upload in one batch; idempotent by record ID so retries are safe.
Any service ← token	Stateless verification	JWT (EdDSA)	No call to Auth is needed per request; services verify tokens locally with Auth's public key.

3.5 Offline-First Design

The Verifier is designed so that its core task has no network dependency at all:

- **What is cached:** the issuer public keys, the latest revocation list (compressed, NFR-C3) with its download time, the agent's seven-day token, and the queue of verification records.
- **Freshness rules (FR-21):** the verifier warns when its list is older than seven days and refuses acceptances after 30 days. These rules bound the revocation gap.
- **No conflicts by construction:** verification records are append-only facts, each with a unique random record ID. Upload is idempotent, so a batch interrupted half-way can simply be resent; duplicates are ignored. There is nothing to merge.
- **Application shell:** the service worker caches the built application, so the screens open with no network after the first visit.
- **Fail securely when the cache is unreadable:** the verifier treats itself as never synchronised and refuses to accept anyone until it syncs.

4. DATA DESIGN

4.1 Data Model

Figure 4.1 shows the entity–relationship diagram. Each service owns its own database; relationships that cross a service boundary are logical (by identifier), never foreign keys, which keeps the services independently deployable.

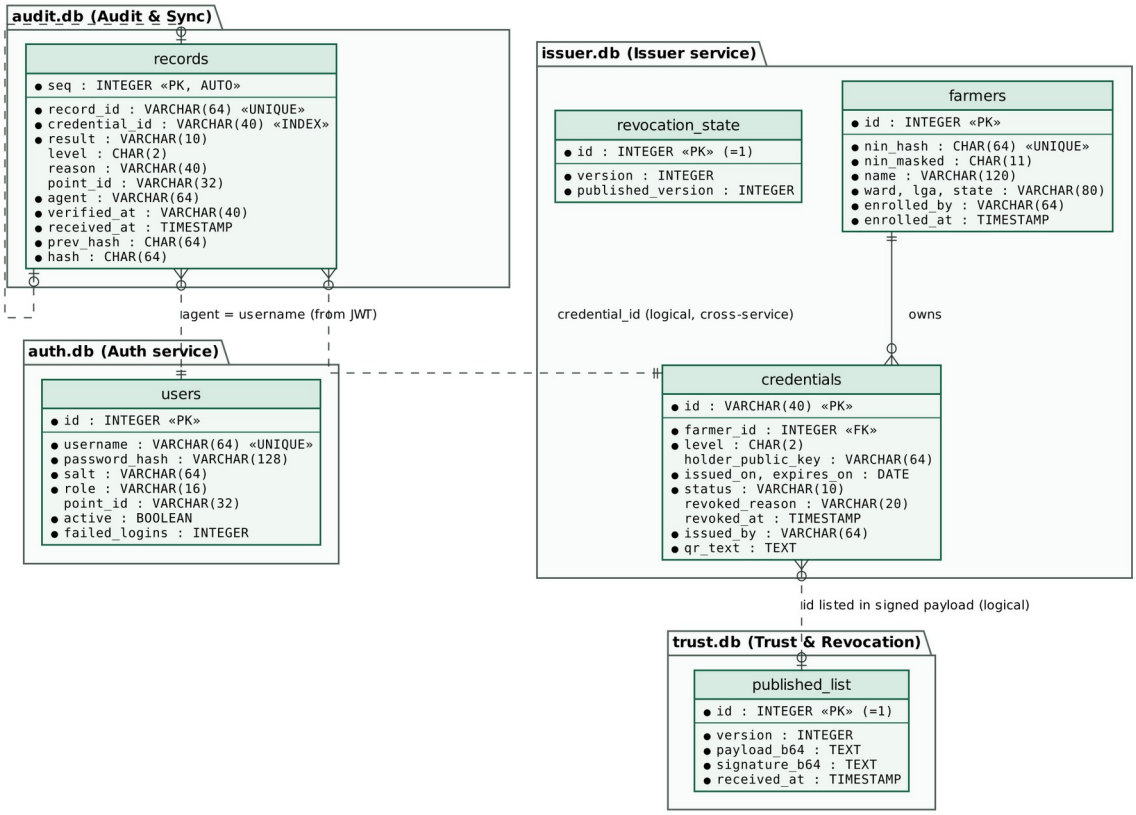


Figure 4.1: Entity–relationship diagram, one database per service

Three integrity rules are enforced in the Issuer: a credential is created only for an enrolled farmer (foreign key); at most one credential per farmer is Active (enforced in the issuing transaction); and an L2 credential carries a valid 32-byte holder public key (validated on input). The audit table enforces append-only integrity by hash chaining rather than by database constraints, so that tampering by someone with database access is still detectable.

4.2 Data Dictionary

The most important fields are described in Table 4.1.

Table 4.1: Data dictionary (selected fields)

Entity.field	Type	Meaning and rules
farmers.nin_hash	CHAR(64), unique	HMAC-SHA256 of the NIN under a secret pepper; detects duplicate enrolment without storing the NIN (D13)
farmers.nin_masked	CHAR(11)	First and last three digits only, e.g. 123*****901 (NFR-S4)
credentials.id	VARCHAR(40), PK	"cred-" + 16 random hex digits; also the ID inside the signed credential
credentials.status	VARCHAR(10)	Active, Revoked or Expired (state machine in QRW-SRS-01 Figure 3.2)
credentials.qr_text	TEXT	The exact signed QR string issued, kept for reprinting a paper wallet
revocation_state.version	INTEGER	Increases on every revocation; published_version shows whether Trust has the latest
published_list.payload_b64	TEXT	The Issuer-signed list exactly as received; Trust never re-signs it
records.record_id	VARCHAR(64), unique	Random ID created on the phone; makes uploads idempotent
records.agent, point_id	VARCHAR	Taken from the uploader's signed token, never from the request body (STRIDE R1)
records.prev_hash, hash	CHAR(64)	hash = SHA-256(prev_hash ‖ "|" ‖ canonical JSON of the record); the first record uses 64 zeros
users.password_hash, salt	VARCHAR	scrypt (N = 2^14, r = 8, p = 1) with a per-user 128-bit salt

4.3 Credential and Message Formats

The signed credential is compact JSON with short field names to keep the QR small (Table 4.2). The Issuer signs the exact bytes; the verifier checks the signature over those same bytes before it reads any field.

Table 4.2: Credential payload fields

Field	Meaning	Example
typ	Message type (domain separation)	QRW1-CRED
id	Credential ID	cred-1422ba201c77b4bf
kid	Issuer key ID (first 12 hex digits of SHA-256 of the public key)	iss-06933dde2eff
lvl	Assurance level	L2
nin	Masked NIN	123*****901
nm, wd, lga, st	Name, ward, LGA, state	Musa Ibrahim, Kura Ward, Kura, Kano
iat, exp	Issue and expiry dates (ISO 8601)	2026-11-03, 2027-11-03
hpk	Holder public key, base64url (L2 only)	PZD6hj0z…
ph	Optional photo thumbnail, base64url JPEG ≤ 1,400 characters	/9j/4AAQ…

The QR text is built in four steps: (1) concatenate the 64-byte Ed25519 signature and the payload bytes; (2) compress with zlib; (3) encode with Base45 (RFC 9285), which fits the QR alphanumeric mode; (4) prefix "QRW1:". A typical credential without a photo is 362 characters. Decoding reverses these steps and refuses any payload that inflates beyond 8 KB, which defeats decompression-bomb codes. The other signed messages are distinguished by their own types: the revocation list ("QRW1-REVOCATIONS") and the challenge message, which is the JSON array ["QRW1-CHALLENGE", nonce, credential ID]. A signature over one type can therefore never be replayed as another.

5. COMPONENT DESIGN

5.1 Cryptographic Core Library

The core library holds every security decision as pure functions with no network or database access, which is what made it possible to build it test-first and to prove its properties with property-based tests. Its modules are listed in Table 5.1.

Table 5.1: qrw_core modules

Module	Main functions	Responsibility
encoding	b45encode/decode, b64url, canonical_json	Wire encodings (RFC 9285 test vectors)
keys	key_id, public_key_b64, save/load_private_key	Ed25519 key handling; issuer key encrypted at rest
credential	issue_credential, verify_credential, mask_nin	Signing and fail-secure verification
qr	encode_qr, decode_qr	QR text format with size guard
challenge	new_challenge, sign_challenge, verify_challenge_response	Offline holder binding
revocation	sign_revocation_list, verify_revocation_list	Signed, versioned revocation list
audit	append_record, record_hash, verify_chain	Tamper-evident hash chain

5.1.1 Verification algorithm

The verification function is the heart of the system. It is written so that every path that is not a complete success ends in a rejection, and so that no field is trusted before the signature has been checked:

```
verify_credential(signed, trusted_keys, revoked_ids, today):
  body ← parse JSON(signed.payload)      fail → REJECT MALFORMED
  key ← trusted_keys[body.kid]           none → REJECT UNKNOWN_ISSUER
  Ed25519.verify(key, signature, payload) bad → REJECT BAD_SIGNATURE
  payload ← strict schema check(body)    fail → REJECT MALFORMED
  reasons ← []
  if today < payload.issued_on:  add NOT_YET_VALID
  if today > payload.expires_on: add EXPIRED
  if payload.id in revoked_ids:  add REVOKED
  return ACCEPT(payload) if reasons empty, else REJECT(reasons)
```

The browser implementation (web/src/lib/qrw.ts) follows the same steps and is checked against vectors produced by the Python implementation, including byte-identical challenge signatures.

5.1.2 Atomic re-issue and publication

When a farmer who already has an active credential is issued a new one, the Issuer revokes the old credential, increments the revocation version and saves the new credential

inside one database transaction. Only after the transaction commits does it sign the full revocation list and push it to Trust. If Trust is unreachable, the issuance still succeeds, the response reports that publication is pending, and the list is republished on the next revocation or on request. The database is therefore always consistent, and the published list catches up.

5.2 Back-End Services

Each service is a FastAPI application created by a factory function that receives its settings and, for the Issuer, its HTTP clients. This lets the tests start all five services in one process with temporary databases and keys, and wire the Issuer to in-process copies of NIMC and Trust. Shared concerns live in one package: settings, database setup, token verification and role checks, rate limiting, security headers, compression and the safe error handler.

5.3 Web Application

The web application is one React and TypeScript code base with a minimal internal router for its fixed screens. Its main design elements are:

- **Verifier:** implements the verification state machine (QRW-SRS-01 Figure 3.1) as explicit steps; stores its cache through a compact, compressed form; enforces list-age rules; queues records.
- **Wallet:** each profile's 32-byte Ed25519 seed is sealed with AES-GCM under a key derived from the PIN by PBKDF2 with 210,000 iterations; five wrong PINs lock the profile for five minutes.
- **Shared library:** the same verification library is used by the Verifier, the Wallet and the Guided Demo, so the demonstration exercises the production code path.
- **Scanner:** camera scanning with a paste fallback, so the system can be demonstrated and tested on a single laptop.

6. INTERFACE DESIGN

All services use REST with JSON bodies; each publishes OpenAPI documentation at /docs. Errors use standard HTTP status codes, summarised in Table 6.1; error bodies contain only a short message, never internal details.

Table 6.1: Error responses

Status	Meaning in this system
401	Missing, forged or expired token; wrong password; missing service key
403	Valid token but the role is not permitted
404	NIN or credential not found
409	Conflict: credential already revoked; revocation list rollback refused; username taken
422	Input failed validation (for example a NIN that is not 11 digits, an oversized photo or batch)
423	Account locked after five failed logins
429	Rate limit exceeded
500	Internal error (generic message only; details logged on the server)

Staff tokens carry the claims in Table 6.2.

Table 6.2: Token claims

Claim	Content
iss	qrw-auth
sub	Username
role	Officer, Agent, Admin or Auditor
pt	Distribution point (agents only)
iat, exp	Issue and expiry time: 8 hours for staff, 7 days for agents

7. SCALABILITY AND RESILIENCE

7.1 Scalability

- **Verification scales with the number of phones, not the server.** Each check runs locally in about 5 ms in the browser library (measured). Adding distribution points adds no verification load to the services.
- **Server load is driven by synchronisation**, which is batched and infrequent: one list download and one record upload per phone per sync. A 100,000-entry revocation list is 0.94 MB compressed (measured), and a 2,000-record batch is accepted in one request.
- **Read-mostly data is cacheable.** The public keys and the signed revocation list are the same for every verifier and are signed, so they can be served from any cache or content delivery network without trusting it.
- **Services are stateless apart from their databases** and can be run as several instances behind a load balancer once SQLite is replaced by PostgreSQL. The in-memory login rate limiter would then move to a shared store.
- **Future growth path:** delta revocation lists instead of full lists, and federated issuers per state, each with its own key ID in the same key set.

7.2 Resilience

Table 7.1 lists the failure modes considered and how the design responds.

Table 7.1: Failure modes and responses

Failure	Effect	Design response
No network at a distribution point	None on verification	Offline-first; records queued and uploaded later
Trust service down	Verifiers cannot refresh	Verifiers use their cached list within the age limits; Issuer republishes when Trust returns
Audit service down	Records cannot upload	Records stay queued on the phone; idempotent retry
Upload interrupted part-way	Some records stored	Re-send the whole batch; duplicates ignored by record ID
NIMC unreachable	No new enrolments	Officer is told to retry; existing credentials unaffected
Stale revocation list offered	Could re-admit revoked credentials	Trust and verifier refuse older versions (anti-rollback)
Malformed or malicious QR	Could crash a verifier	Size guard and fail-secure parsing; property-tested on random input
Service crash	Request fails	Container restart policy; generic error message; no partial transactions

8. SECURITY DESIGN SUMMARY

Security is designed in, not added. The main measures are summarised here; the threat model and test evidence are in QRW-SEC-03.

- **Least privilege:** only the Issuer reads the signing key; services hold only public keys for tokens; each endpoint names the roles allowed.
- **Defence in depth:** four independent checks on every credential (issuer signature, holder challenge, revocation and expiry, human face check).
- **Fail securely:** every malformed, unknown or expired input is a rejection; server errors return a generic message.
- **Data minimisation:** masked NIN in credentials; keyed hash in the database; no BVN or telephone number anywhere.

9. DESIGN TRACEABILITY

Table 9.1 maps each requirement group to the design elements that realise it. Requirement-level traceability to code and tests is in the RTM (QRW-SRS-01, Appendix A).

Table 9.1: Requirements to design elements

Requirements	Design elements
FR-01 to FR-07	Issuer service; credential and revocation_state tables; atomic re-issue (Section 5.1.2)
FR-08, FR-09	Trust service; signed revocation list format; anti-rollback
FR-10 to FR-13	Wallet screen; PIN vault; challenge signing
FR-14 to FR-23	Verifier screen and cache; verification algorithm (Section 5.1.1); QR formats
FR-24 to FR-26	Audit service; hash chain; anomaly query
FR-27 to FR-29	Auth service; token claims; role checks
NFR-P1 to NFR-P3	Edge verification; asynchronous synchronisation; failure-mode responses
NFR-S1 to NFR-S8	Security design summary; QRW-SEC-03
NFR-C1 to NFR-C4	Compact credential encoding; compressed list; batch cap